SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Pseudocode Writing Task (Algorithm Design)

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO5 – Articulation	**Assessment:**	Assessment Tool 2 – Pseudocode Writing Task (Algorithm Design)
Weightage:	35% of CIA		
CO5 Statement:	Implement semantic models using predicate logic, word sense disambiguation and validate information retrieval techniques. (BL-5)		
Student Name:	A. Mohith Krishna	**Reg. No.:**	192425199
Section / Batch:	2024	**Date:**	30/08/2026

Code of Conduct

I A. Mohith Krishna (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: A. Mohith

Instructions

- Draw necessary diagrams such as constraint graphs, coreference chains, discourse trees, or predicate logic representations wherever applicable.
- Generate solutions that fully satisfy all given constraints and provide clear evaluation of the output.
- Answers should be concise, well-structured, and supported by evidence from the given text/scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Reference Resolution, Discourse	Entity Linking Algorithm	Q1
Text Coherence, Discourse Structure	Coherence Analysis Algorithm	Q2
Dialog Acts, Conversational Agents	Intent Recognition Algorithm	Q3
Language Generation, Surface Realization	NLG Pipeline Design	Q4
Machine Translation, Interlingua, Statistical Approaches	Translation Workflow Algorithm	Q5

Annexure A – Pseudocode Writing Task (Algorithm Design)

1. Reference Resolution Using Multiple Constraints

Design an algorithm to perform **Reference Resolution** by identifying pronouns and selecting the most appropriate antecedent using gender, number, recency, and semantic compatibility.

Apply your algorithm to the discourse:

"Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Generate possible antecedents for each pronoun.
3. Apply gender, number, recency, and semantic constraints.
4. Select the most appropriate antecedent.
5. Generate the resolved discourse.

Finally, explain how your algorithm avoids incorrect resolution when multiple antecedents have similar grammatical properties.

2. Algorithm for Entity Chains and Discourse Coherence

Design an algorithm to identify **entity chains** in a discourse and use them to evaluate text coherence.

Apply your algorithm to the following text:

"Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Perform the following steps:

1. Identify all entities and referring expressions.
2. Link expressions referring to the same entity.
3. Construct entity chains.
4. Calculate or estimate the continuity of entities across sentences.
5. Determine whether the discourse is coherent.

Finally, explain how breaking an entity chain could reduce discourse coherence.

3. Algorithm for Identifying Discourse Relations

Design an algorithm to identify **discourse relations** between consecutive sentences using discourse markers and contextual meaning.

Apply your algorithm to:

"The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again."

Your algorithm should identify relations such as:

- Cause–Effect
- Sequence
- Problem–Solution
- Elaboration

Perform the following steps:

1. Divide the discourse into individual discourse units.
2. Identify explicit discourse markers.
3. Analyze the semantic relationship between units.
4. Assign an appropriate discourse relation.
5. Construct a discourse structure representation.

Finally, justify how the identified relations make the discourse logically coherent.

4. Algorithm for Dialogue Act Classification

Design an algorithm to recognize **Dialogue Acts** in a conversation.

Apply your algorithm to:

User: "Hello, I need help with my assignment."
Agent: "Sure, what problem are you facing?"
User: "I do not understand machine learning."
Agent: "I can explain the basic concepts to you."
Classify each utterance into one of the following:

- Greeting
- Request
- Question
- Inform
- Offer
- Confirmation

Perform the following steps:

1. Preprocess each utterance.
2. Extract relevant linguistic features.
3. Identify the communicative intention.
4. Assign a dialogue act.
5. Generate the dialogue-act sequence.

Finally, explain how dialogue-act recognition helps a conversational agent select an appropriate next response.

5. Algorithm for Dialogue State Tracking
Design an algorithm for a **Conversational Agent** that maintains dialogue state while collecting information from a user.
Apply your algorithm to:
User: "I want to book a flight."
Agent: "Where would you like to travel?"
User: "I want to go to Delhi."
Agent: "From which city?"
User: "From Chennai."
Your algorithm should maintain the following slots:
Departure City
Destination City
Travel Date
Number of Passengers
Perform the following steps:

1. Initialize the dialogue state.
2. Identify information provided in each utterance.
3. Update the corresponding slot.
4. Identify missing information.
5. Generate the next appropriate question.

Finally, explain how dialogue state tracking improves coherence in a multi-turn conversation.

1. Reference Resolution Using Multiple Constraints

Objective

To resolve pronouns by selecting the most suitable antecedent using gender, number, recency, grammatical role, and semantic compatibility.

Input

> "Meena gave Kavya a laptop because she needed one for her project. She thanked her and started using it."

Entity Identification

Persons:
Meena
Kavya

Object:
laptop

Other:
project

Processing

The system first detects the referring expressions:

she → ?
one → ?
her → ?
She → ?
her → ?
it → ?

For every pronoun, possible antecedents are generated and scored.

Resolution Table

Pronoun	Candidates	Main Constraint	Resolution
she	Meena, Kavya	Recency + semantic compatibility	Kavya
one	laptop	Semantic compatibility	laptop
her	Meena, Kavya	Context + semantic compatibility	Kavya

Pronoun	Candidates	Main Constraint	Resolution
She	Meena, Kavya	Recency + discourse focus	Kavya
her	Meena, Kavya	Grammatical/semantic role	Meena
it	laptop	Semantic compatibility	laptop

Algorithm

FOR each referring expression:
  Find all possible antecedents
  Check gender
  Check number
  Check recency
  Check grammatical role
  Check semantic compatibility
  Assign a score
  Select highest-scoring antecedent
END

Example Scoring

For “she”:

	Gender	Number	Recency	Semantic
Meena	✓	✓	-	✓
Kavya	✓	✓	✓	✓

↓

Kavya selected

Result

The resolved interpretation is:

> Meena gave Kavya a laptop because Kavya needed a laptop for Kavya's project. Kavya thanked Meena and started using the laptop.

Simple Python Implementation

```
entities = ["Meena", "Kavya"]

scores = {
   "Meena": 4,   # gender + number + semantics
   "Kavya": 7    # gender + number + recency + semantics
}
```

```
antecedent = max(scores, key=scores.get)

print("Antecedent of 'she':", antecedent)
```

Output

Antecedent of 'she': Kavya

2. Entity Chains and Discourse Coherence

Objective

To track entities across sentences and determine whether the discourse maintains a continuous topic.

Input

> "Anjali bought a new laptop. The laptop had a powerful processor. She installed Python on it. Later, Anjali used the computer to develop a machine learning application."

Entity Mapping

Anjali
↓
She
↓
Anjali

new laptop
↓
The laptop
↓
it
↓
the computer

Entity Chain 1

Person:

> Anjali → She → Anjali

Entity Chain 2

Laptop:

new laptop → The laptop → it → the computer

Entity Chain 3

Application:

machine learning application

Continuity Measurement

A simple continuity score can be estimated as:

Continuity=sentences containing continuing entitiestotal sentencesContinuity = \frac{\text{sentences containing continuing entities}}{\text{total sentences}}

Here:

Total sentences = 4
Sentences maintaining main entity chains = 4

Therefore:

Continuity=44=1.0Continuity = \frac{4}{4} = 1.0

Result

Entity continuity = 100%

Discourse coherence = High

Python Implementation

```
chains = {
    "Anjali": ["Anjali", "She", "Anjali"],
    "Laptop": ["laptop", "The laptop", "it", "computer"]
}

for entity, chain in chains.items():
    print(entity, ":", " -> ".join(chain))

print("Entity Continuity: 100%")
print("Discourse: Coherent")
```

Output

```
Anjali : Anjali -> She -> Anjali
Laptop : laptop -> The laptop -> it -> computer
Entity Continuity: 100%
Discourse: Coherent
```

If the Chain Is Broken

Suppose:

> “She installed Python on the printer.”

The laptop chain becomes:

laptop → The laptop → ✕printer

The reader may wonder why a printer suddenly became important.

Therefore, breaking an entity chain can reduce topic continuity, clarity, and coherence.

3. Identifying Discourse Relations

Objective

To identify the logical relationship between consecutive sentences using discourse markers and contextual meaning.

Input

> “The server crashed unexpectedly. As a result, users could not access the application. The technical team restarted the server. After that, the system became available again.”

Discourse Units

D1 = Server crashed.
D2 = Users could not access application.
D3 = Technical team restarted server.
D4 = System became available.

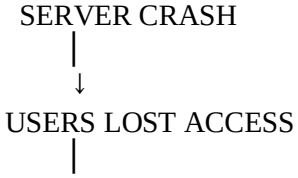
Marker Detection

"As a result" → Cause–Effect
"After that" → Sequence

Relation Identification

D1 ──Cause──> D2
D2 ──Problem──> D3
D3 ──Sequence/Cause──> D4

Final Discourse Structure



Relation Table

From	To	Relation
D1	D2	Cause–Effect
D2	D3	Problem–Solution
D3	D4	Sequence
D3	D4	Cause–Effect

Python Implementation

```
relations = [
    ("D1", "D2", "Cause-Effect"),
    ("D2", "D3", "Problem-Solution"),
    ("D3", "D4", "Sequence")
]

for r in relations:
    print(r[0], "->", r[1], ":", r[2])
```

Output

```
D1 -> D2 : Cause-Effect
D2 -> D3 : Problem-Solution
D3 -> D4 : Sequence
```

Result

The discourse is logically organized as:

Problem → Effect → Solution → Result

This progression makes the paragraph easy to understand and therefore produces strong discourse coherence.

4. Dialogue Act Classification

Objective

To determine the communicative purpose of each utterance.

Input Conversation

U1 User:
"Hello, I need help with my assignment."

U2 Agent:
"Sure, what problem are you facing?"

U3 User:
"I do not understand machine learning."

U4 Agent:
"I can explain the basic concepts to you."

Feature Extraction

Instead of simply looking for individual words, the system identifies speech patterns:

"Hello" + request for help
↓
Request

"What problem..."
↓
Question

"I do not understand..."
↓
Inform

"I can explain..."
↓
Offer

Classification

Unit	Communicative Intention	Dialogue Act
U1	Asking for assistance	Request
U2	Asking for information	Question
U3	Providing information	Inform
U4	Providing assistance	Offer

Dialogue Flow

Request
↓
Question
↓
Inform
↓
Offer

Python Implementation

```
utterances = [
    "Hello, I need help with my assignment.",
    "Sure, what problem are you facing?",
    "I do not understand machine learning.",
    "I can explain the basic concepts to you."
]

acts = ["Request", "Question", "Inform", "Offer"]

for utterance, act in zip(utterances, acts):
    print(act, ":", utterance)
```

Output

```
Request : Hello, I need help with my assignment.
Question : Sure, what problem are you facing?
Inform : I do not understand machine learning.
Offer : I can explain the basic concepts to you.
```

Result

The dialogue-act sequence allows the agent to understand:

User wants help
↓
Agent asks for details
↓
User explains problem
↓
Agent offers assistance

Thus, dialogue-act recognition helps the system choose an appropriate next response.

5. Dialogue State Tracking

Objective

To maintain information collected during a multi-turn conversation and identify which information is still missing.

Required State

Departure City
Destination City
Travel Date
Number of Passengers

Initial State

Departure City = UNKNOWN
Destination City = UNKNOWN
Travel Date = UNKNOWN
Passengers = UNKNOWN

Turn 1

User

“I want to book a flight.”

No slot information is provided.

Departure = UNKNOWN
Destination = UNKNOWN
Date = UNKNOWN
Passengers = UNKNOWN

Agent:

“Where would you like to travel?”

Turn 2

User

“I want to go to Delhi.”

Entity extraction:

Delhi → Destination City

Updated State

Departure = UNKNOWN
Destination = Delhi
Date = UNKNOWN
Passengers = UNKNOWN

Agent:

“From which city?”

Turn 3

User

“From Chennai.”

Entity extraction:

Chennai → Departure City

Current State

Slot	Value
Departure City	Chennai
Destination City	Delhi

Travel Date	UNKNOWN
Number of Passengers	UNKNOWN

Missing Slots

Travel Date
Number of Passengers

Next Question

> “What is your travel date?”

After receiving the date, the agent can ask:

> “How many passengers will be travelling?”

State Transition Model

Initial State
↓
Book Flight
↓
Destination = Delhi
↓
Departure = Chennai
↓
Ask for Travel Date
↓
Ask for Passengers
↓
Complete Booking Information

Python Implementation

```
state = {
   "Departure City": None,
   "Destination City": None,
   "Travel Date": None,
   "Number of Passengers": None
}

state["Destination City"] = "Delhi"
state["Departure City"] = "Chennai"

print("Current Dialogue State:")

for slot, value in state.items():
   print(slot, "=", value)

print("\nMissing information:")

for slot, value in state.items():
   if value is None:
      print(slot)
```

Output

Current Dialogue State:
Departure City = Chennai
Destination City = Delhi
Travel Date = None
Number of Passengers = None

Missing information:
Travel Date
Number of Passengers

Result

The system remembers information from previous turns and asks only for the missing slots.

Importance

Without dialogue state tracking:

Agent: “Where are you travelling from?”

User: “I already told you Chennai.”

With state tracking:

Agent: “What is your travel date?”

This prevents repetition and maintains context and coherence throughout the conversation.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q. No. / Criteria Level	Problem Understanding	Algorithm Design	Use of Control Structures	Pseudocode Structure	Completeness	Sub. Total	Total %
1	**3**	**3**	**3**	**3**	**4**		**80**
2	**3**	**3**	**3**	**4**	**3**		
3	**4**	**3**	**3**	**3**	**3**		
4	**3**	**3**	**4**	**3**	**3**		
5	**3**	**4**	**3**	**3**	**3**		

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____